# Lab Report

Operating Systems Laboratory
Assignment 4: Memory Management

Anshuman Rahul (CS24BT060)
Sidharth B (CS24BT061)

## 1 Setup and Assumptions

### 1.1 Simulated system

The simulator models a 32-bit virtual address space with 4 KB pages, so an address splits into a 20-bit virtual page number (VPN) and a 12-bit offset, giving $2^{20}$ virtual pages. Only the lower 32 bits of each trace address are used (the traces contain addresses wider than 32 bits). The TLB has 128 entries, is fully associative with LRU replacement, and is empty at the start of every run. Instruction/data and read/write flags in the traces are not used.

### 1.2 Cost model

Every trace line is one memory access of the benchmark. Let $w$ be the number of memory accesses needed to walk the page table ($w = 1$ for the single-level table, $w = 2$ for the two-level table). The cycles charged per access are:

| Case | Cycles |
|---|---|
| No TLB | $200\,w + 200$ |
| TLB hit | $1 + 200$ |
| TLB miss | $1 + 200\,w + 200$ |
| Page replacement (extra, Part IV only) | +20,000,000 |

The final 200 cycles is the access to the data itself. A page fault into a free frame costs the same as any page-table walk and no disk time, since the first pages come for free; disk time is charged only when a resident page has to be replaced. We do not model where the page table itself is stored in physical memory: page-table pages do not occupy frames, and only the cost of walking them is charged.

### 1.3 Frame management scheme

- **Parts I–III** (more frames than pages): frames are handed out sequentially, $0, 1, 2, \ldots$, in the order pages are first touched, and are never freed. The number of frames used is therefore the number of distinct pages touched.
- **Part IV** (limited frames): frames are initially handed out the same way. Once all frames are in use, a page fault evicts the least recently used page. Recency is kept in a doubly linked list over the frames; every access (TLB hit or not) moves the accessed page's frame to the most-recently-used end, and the victim is the frame at the least-recently-used end. On eviction the simulator removes the victim from the page table, invalidates its TLB entry (so the TLB can never return a translation to a frame that has been reused), charges the disk cost, and gives the freed frame to the faulting page.

### 1.4 TLB and LRU implementation

- **TLB.** A flat array of 128 entries, each holding a VPN, a frame number, a valid bit and the value of a global access counter at its last use. A lookup scans the array linearly and, on a hit, refreshes the entry's counter. An insertion uses the first invalid entry if there is one and otherwise replaces the entry with the smallest counter (the least recently used). Invalidation marks the matching entry invalid.
- **Frame LRU list.** The doubly linked list is stored in arrays indexed by frame number (with a head and a tail sentinel), so the list node of a frame is found directly. Moving a frame to the most-recently-used end and evicting the least-recently-used frame are both constant-time operations; a frame that is already the most recently used is left in place.

### 1.5 Page tables

The *single-level* table is an array of $2^{20}$ entries indexed by the VPN. The *two-level* table splits the VPN into a 10-bit directory index and a 10-bit second-level index. Second-level tables (1024 entries each) are allocated only when the first page they cover is mapped, and are freed when their last valid entry is removed. Each second-level table therefore covers 1024 pages (a 4 MB region of the address space).

### 1.6 Statistics

**A. Frames used** the peak number of frames in use.

**B. Hot pages** the number of accesses to every virtual page, from which the cumulative access share of the hottest pages is computed.

**C. Cycles** total cycles over the trace, using the cost model above.

**D. Page table utilization** number of distinct pages accessed divided by the number of page table entries allocated. For the single-level table the denominator is $2^{20}$. For the two-level table it is $1024\times$ (number of live second-level tables); the fixed 1024-entry top-level directory is *not* counted.

**E. Hash** $H \leftarrow (H \oplus \mathit{frame}) \times 1099511628211 \pmod{2^{64}}$, applied once per access with $H = 0$ initially.

### 1.7 Hash function

The hash of a benchmark is a single 64-bit value computed over the whole run. It starts at $H = 0$, and every access to memory (a TLB hit, a page-table hit or a page fault alike) updates it with the frame number $f$ that the access used:

$$H \leftarrow (H \oplus f) \times 1099511628211 \pmod{2^{64}}.$$

The constant is the 64-bit FNV prime; the update is a variation on the FNV-1a hash, which XORs in one byte at a time, whereas here a whole frame number is XORed in. The properties we rely on are the following.

- **Deterministic.** $H$ depends only on the sequence of frame numbers, so two runs that use the same frame for every access, in the same order, end with the same value. This is why the single-level and two-level simulators must agree (Part II): the frame allocator hands out frames in first-touch order and does not depend on the page table.
- **Order-sensitive.** The update is not commutative: applying frames $f_1$ then $f_2$ gives $((f_1 p) \oplus f_2)\, p$, whereas $f_2$ then $f_1$ gives $((f_2 p) \oplus f_1)\, p$, which are different in general. The hash therefore reflects the order in which frames are used, not just which frames are used.

- **No information lost at each step.** The multiplier is odd, so multiplication modulo $2^{64}$ is a bijection, and so is XOR with a fixed $f$. For a given frame number, two different values of $H$ can never become equal after one update.

The hash is a checksum, not a proof. If two runs end with different values, their frame sequences certainly differ, because the computation is deterministic. If they end with the same value, the sequences are very likely but not provably identical, since a 64-bit value cannot distinguish every pair of sequences; we do not quantify that probability.

**How the values are reported.** For each benchmark we report the final value of $H$ after the last access of the trace, as an unsigned 64-bit integer written as 16 hexadecimal digits. The single-level value is in the last column of Table 1 (Part I) and the two-level value in the last column of Table 2 (Part II), so the two can be compared row by row. The programs also compare the two values themselves: Part II compares the two-level run with a single-level run of the same trace, and Part III compares the single-level and two-level runs with the TLB; both print "yes" or "NO" in a `hash_match` column. The hash is not reported for Part IV, where replacement changes which frames pages use.

### 1.8 Environment

The programs are compiled with g++ 13.3.0 using `-std=c++17 -O2`. The ten benchmark traces contain between 11.5 and 12.9 million accesses each. Traces are streamed from disk in 1 MB chunks and parsed by hand.

## 2 Part I: Single-Level Page Table

Table 1 shows the results with a single-level table, no TLB, and enough frames that no page is ever evicted.

Table 1: Part I results (single-level page table, no TLB). "Top-10% share" is the fraction of all accesses that go to the most-accessed 10% of the pages the benchmark touches. "Hash" is the final value of $H$ (64-bit, hexadecimal).

| Benchmark | Accesses (M) | Frames used | PT util. (%) | Top-10% share (%) | Cycles ($10^9$) | Hash |
|---|---|---|---|---|---|---|
| blender | 12.37 | 1551 | 0.1479 | 99.2 | 4.948 | `e63e5f066c37e99e` |
| fotonik3d | 12.89 | 1602 | 0.1528 | 91.3 | 5.156 | `c62619cfcf931b0c` |
| gcc | 12.47 | 990 | 0.0944 | 82.5 | 4.989 | `4189fb695f6069f7` |
| imagick | 11.55 | 96 | 0.0092 | 89.1 | 4.620 | `7fa618e7ef7712eb` |
| lbm | 12.84 | 2131 | 0.2032 | 94.1 | 5.137 | `6e17e34e57b8975d` |
| mcf | 12.47 | 1597 | 0.1523 | 97.9 | 4.990 | `9e61e23f117ee4a4` |
| perlbench | 12.57 | 630 | 0.0601 | 89.3 | 5.027 | `72095b979695151d` |
| x264 | 11.72 | 234 | 0.0223 | 90.0 | 4.689 | `9da6f2e91ac155a1` |
| xalancbmk | 11.97 | 14325 | 1.3661 | 95.2 | 4.789 | `e43e4394367b9c0c` |
| xz | 11.85 | 8037 | 0.7665 | 99.5 | 4.741 | `49965a8c8a4160b2` |

**Frames used (A).** Each benchmark touches only 96 to 14 325 distinct pages out of the $2^{20} =$ 1,048,576 available, so that many frames are used.

**Hot pages (B).** Figure 1 plots, for each benchmark, the cumulative share of accesses covered by the hottest pages (pages sorted by access count, hottest first). In every benchmark the hottest 10% of touched pages account for 82.5%–99.5% of all accesses (Table 1), which supports the hypothesis that a small fraction of pages sees a large fraction of the accesses. The fractions in this figure are of the pages each benchmark *touches*, not of the whole address space.

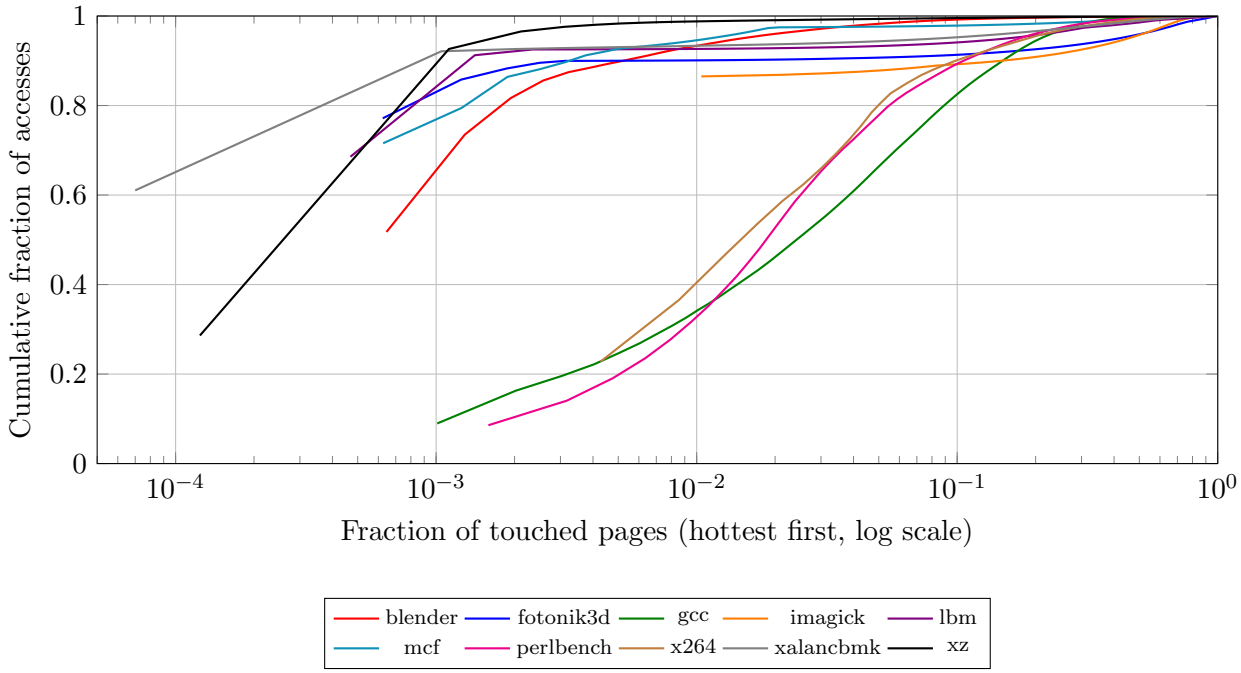



Figure 1: Cumulative access share of the hottest pages for each benchmark (Part I). Each curve is sampled at log-spaced points.

**Page table utilization (D).** Only 0.009%–1.37% of the entries of the single-level table are ever accessed. The table always has $2^{20}$ entries allocated, whatever the benchmark touches.

**Cycles (C).** With no TLB, every access costs 400 cycles (one table access plus the data access), so total cycles are exactly 400× accesses (4.620–5.156 billion cycles).

**Hash (E).** The last column of Table 1 lists the final hash of each benchmark with the single-level table. Table 2 in Part II lists the hashes produced with the two-level table, so the two can be compared row by row.

## 3 Part II: Two-Level Page Table

Table 2: Part II results (two-level page table, no TLB), with the single-level numbers for comparison. "L2 tables" is the number of second-level tables allocated. The frames used are identical to Part I, and the hash column (two-level table) can be compared directly with the last column of Table 1 (single-level table).

| Benchmark | L2 tables | Entries alloc. (2L) | Util. 2L (%) | Util. 1L (%) | Cycles 2L ($10^9$) | Cycles 1L ($10^9$) | Hash (2L) |
|---|---|---|---|---|---|---|---|
| blender | 47 | 48 128 | 3.22 | 0.1479 | 7.422 | 4.948 | `e63e5f066c37e99e` |
| fotonik3d | 14 | 14 336 | 11.17 | 0.1528 | 7.734 | 5.156 | `c62619cfcf931b0c` |
| gcc | 70 | 71 680 | 1.38 | 0.0944 | 7.484 | 4.989 | `4189fb695f6069f7` |
| imagick | 5 | 5 120 | 1.88 | 0.0092 | 6.929 | 4.620 | `7fa618e7ef7712eb` |
| lbm | 6 | 6 144 | 34.68 | 0.2032 | 7.706 | 5.137 | `6e17e34e57b8975d` |
| mcf | 13 | 13 312 | 12.00 | 0.1523 | 7.485 | 4.990 | `9e61e23f117ee4a4` |
| perlbench | 48 | 49 152 | 1.28 | 0.0601 | 7.541 | 5.027 | `72095b979695151d` |
| x264 | 11 | 11 264 | 2.08 | 0.0223 | 7.033 | 4.689 | `9da6f2e91ac155a1` |
| xalancbmk | 54 | 55 296 | 25.91 | 1.3661 | 7.183 | 4.789 | `e43e4394367b9c0c` |
| xz | 43 | 44 032 | 18.25 | 0.7665 | 7.111 | 4.741 | `49965a8c8a4160b2` |

**Hot pages (B).** The per-page access counts depend only on the sequence of virtual pages in the trace, not on the page table type, so Figure 1 and the top-10% shares in Table 1 apply unchanged to the two-level simulator.

**Hash.** The hash computed with the two-level table is identical to the one from Part I (last column of Table 1 against the last column of Table 2) for all ten benchmarks. This is expected: the hash depends only on the sequence of frame numbers, the frame allocator is the same in both simulators, and the page table type does not affect which frame a page gets.

**Space.** The number of pages touched is the same for both table types, so the difference in utilization (Table 2) comes entirely from the denominator. The two-level table allocates only the second-level tables that cover touched pages, namely 5 to 70 tables per benchmark (one per distinct 4 MB region touched, since nothing is evicted), against the fixed $2^{20}$ entries of the single-level table. Its utilization is 1.28%–34.7%, compared with 0.009%–1.37% for the single-level table. The top-level directory (1024 entries) is a fixed cost that is not included in these figures.

**Time.** Each walk of the two-level table needs one more memory access, so every access costs 600 cycles instead of 400, and the total cycles are exactly 1.5 times the single-level total for every benchmark. The two-level table thus trades extra cycles for less allocated page-table space.

# 4 Part III: Adding a TLB

Table 3: Part III results (128-entry fully associative LRU TLB). Speedup is the Part I/II cycle count (no TLB) divided by the cycle count with the TLB, for the same page table type. The TLB hit rate is the same for both page table types.

| Benchmark | TLB hit rate (%) | Cycles 1L ($10^9$) | Cycles 2L ($10^9$) | Speedup 1L | Speedup 2L |
|---|---|---|---|---|---|
| blender | 99.9398 | 2.488 | 2.489 | 1.99 | 2.98 |
| fotonik3d | 99.9867 | 2.591 | 2.592 | 1.99 | 2.98 |
| gcc | 99.8874 | 2.510 | 2.513 | 1.99 | 2.98 |
| imagick | 99.9992 | 2.321 | 2.321 | 1.99 | 2.99 |
| lbm | 99.9789 | 2.582 | 2.583 | 1.99 | 2.98 |
| mcf | 99.4421 | 2.521 | 2.535 | 1.98 | 2.95 |
| perlbench | 99.5484 | 2.538 | 2.549 | 1.98 | 2.96 |
| x264 | 99.9659 | 2.357 | 2.358 | 1.99 | 2.98 |
| xalancbmk | 99.8673 | 2.410 | 2.413 | 1.99 | 2.98 |
| xz | 99.7669 | 2.388 | 2.393 | 1.99 | 2.97 |

**Hit rate (A).** The TLB hit rate is 99.44%–99.9992% across the benchmarks (Table 3); the lowest is mcf. The rate is identical for the single-level and two-level tables, because the TLB logic does not depend on the page table type: both simulators present it with the same stream of virtual page numbers.

**Cycles (B).** With the TLB, total cycles drop to 2.321–2.591 billion. This is a speedup of 1.98–1.99 times for the single-level table and 2.95–2.99 times for the two-level table over the no-TLB runs. The two-level table benefits more because a TLB hit skips the entire page-table walk, and the walk is more expensive for the two-level table. With the TLB, the difference between the two table types shrinks to 200 cycles per TLB miss (one extra memory access per walk), which is a small fraction of the total.

## 5 Temporal and Spatial Locality

The earlier results say something about how the benchmarks use their pages. This section collects that evidence; it is limited to what the measurements above can show, and we did not run any additional locality experiments (for example, we did not measure the distance between consecutive accesses).

Table 4: Page-level footprint of each benchmark. "Accesses per page" is total accesses divided by distinct pages touched. "Regions" is the number of second-level tables allocated in Part II, i.e. the number of distinct 4 MB-aligned regions of the address space that contain at least one touched page (out of 1024 possible). "Pages per region" is pages touched divided by regions. The TLB hit rate is from Part III.

| Benchmark | Pages touched | Accesses per page | Regions | Pages per region | TLB hit rate (%) |
|---|---|---|---|---|---|
| blender | 1 551 | 7 975 | 47 | 33 | 99.9398 |
| fotonik3d | 1 602 | 8 046 | 14 | 114 | 99.9867 |
| gcc | 990 | 12 599 | 70 | 14 | 99.8874 |
| imagick | 96 | 120 300 | 5 | 19 | 99.9992 |
| lbm | 2 131 | 6 027 | 6 | 355 | 99.9789 |
| mcf | 1 597 | 7 811 | 13 | 123 | 99.4421 |
| perlbench | 630 | 19 949 | 48 | 13 | 99.5484 |
| x264 | 234 | 50 091 | 11 | 21 | 99.9659 |
| xalancbmk | 14 325 | 836 | 54 | 265 | 99.8673 |
| xz | 8 037 | 1 475 | 43 | 187 | 99.7669 |

**Temporal locality.** Each benchmark makes between 11.5 and 12.9 million accesses to at most 14 325 distinct pages, so every page is accessed hundreds to over a hundred thousand times on average (Table 4), and the access counts are heavily skewed towards a few pages (Table 1). More directly, in Part III no page is ever evicted, so the TLB is a pure 128-entry LRU cache of translations. An access therefore hits in the TLB exactly when its page is among the 128 most recently used distinct pages. A hit rate of 99.44%–99.9992% thus means that, for nearly every access, the same page was already used within the last 128 distinct pages. We read this as strong temporal locality at page granularity. The benchmarks differ (mcf and perlbench have the lowest hit rates, about 99.4% and 99.5%), but we did not investigate why.

**Spatial locality.** The touched pages are concentrated in a small part of the address space: the benchmarks use only 5 to 70 of the 1024 possible 4 MB regions, i.e. between 0.5% and 6.8%. This clustering is what lets the two-level page table allocate so few second-level tables (Part II). Within the regions that are used, the touched pages do not fill the region densely: the average is between 13 and 355 touched pages per region, out of 1024 (1.3%–34.7%, the two-level utilization of Table 2). So the footprint is clustered at the scale of megabytes, but sparse within the clusters. The page-level view cannot tell us whether consecutive accesses are close to each other in address.

**Implication.** A small footprint with strong page reuse is consistent with the other results: a 128-entry TLB covers most accesses, and Part IV starts replacing pages only when physical memory is smaller than the benchmark's footprint.

## 6 Part IV: Limited Physical Memory

The traces touch at most 14 325 pages, which is less than $2^{14} = 16384$ frames, i.e. less than 1/64 of the virtual address space. Physical memory sizes of 1/2, 1/4, 1/8, 1/16 and 1/32 of the virtual address space therefore give exactly the same result as 1/2: nothing is ever evicted. We ran

physical memory sizes of 1/262144, 1/65536, 1/16384, 1/4096, 1/1024, 1/256, 1/64 and 1/2 of the virtual address space ($2^2$, $2^4$, $2^6$, $2^8$, $2^{10}$, $2^{12}$, $2^{14}$ and $2^{19}$ frames). All runs use the TLB. A page fault is any first load of a page into memory, including loads into a free frame; an eviction is a fault that has to replace a resident page, and only evictions are charged the 20 million cycle disk cost.

Table 5: Part IV: page faults per million accesses for each physical memory size (columns are numbers of frames; the smaller number below each is that many frames as a fraction of the $2^{20}$ virtual pages). The fault counts are identical for the single-level and two-level tables. The 16384-frame column equals the compulsory (first-touch) faults.

| Benchmark | 4<br>$2^{-18}$ | 16<br>$2^{-16}$ | 64<br>$2^{-14}$ | 256<br>$2^{-12}$ | 1024<br>$2^{-10}$ | 16384<br>$2^{-6}$ |
|---|---|---|---|---|---|---|
| blender | 32 489 | 7 382 | 3 699 | 381 | 151 | 125 |
| fotonik3d | 139 129 | 3 103 | 133 | 133 | 133 | 124 |
| gcc | 71 887 | 19 926 | 3 199 | 400 | 79 | 79 |
| imagick | 641 | 581 | 568 | 8 | 8 | 8 |
| lbm | 37 761 | 523 | 211 | 211 | 211 | 166 |
| mcf | 80 078 | 16 953 | 7 046 | 3 253 | 131 | 128 |
| perlbench | 91 255 | 29 318 | 9 100 | 1 620 | 50 | 50 |
| x264 | 14 646 | 1 631 | 539 | 20 | 20 | 20 |
| xalancbmk | 46 800 | 2 082 | 1 381 | 1 252 | 1 251 | 1 197 |
| xz | 42 434 | 10 963 | 2 779 | 2 029 | 1 310 | 678 |

Table 6: Part IV: total cycles ($10^9$) for each physical memory size (columns are numbers of frames; the smaller number below each is that many frames as a fraction of the $2^{20}$ virtual pages), two-level table with TLB. The single-level table gives the same fault and eviction counts and differs only by one extra memory access per TLB miss. The 524288-frame column is 1/2 of the virtual address space.

| Benchmark | 4<br>$2^{-18}$ | 16<br>$2^{-16}$ | 64<br>$2^{-14}$ | 256<br>$2^{-12}$ | 1024<br>$2^{-10}$ | 4096<br>$2^{-8}$ | 16384<br>$2^{-6}$ | 524288<br>$2^{-1}$ |
|---|---|---|---|---|---|---|---|---|
| blender | 8 040.0 | 1 828.4 | 916.3 | 91.6 | 19.5 | 2.5 | 2.5 | 2.5 |
| fotonik3d | 35 870.8 | 802.3 | 35.7 | 31.8 | 16.5 | 2.6 | 2.6 | 2.6 |
| gcc | 17 936.1 | 4 973.3 | 799.2 | 97.3 | 2.5 | 2.5 | 2.5 | 2.5 |
| imagick | 150.3 | 136.3 | 132.2 | 2.3 | 2.3 | 2.3 | 2.3 | 2.3 |
| lbm | 9 702.3 | 136.6 | 55.4 | 51.5 | 36.2 | 2.6 | 2.6 | 2.6 |
| mcf | 19 981.6 | 4 231.8 | 1 759.3 | 809.1 | 14.8 | 2.5 | 2.5 | 2.5 |
| perlbench | 22 940.7 | 7 371.8 | 2 288.8 | 404.7 | 2.5 | 2.5 | 2.5 | 2.5 |
| x264 | 3 435.7 | 384.4 | 127.5 | 2.4 | 2.4 | 2.4 | 2.4 | 2.4 |
| xalancbmk | 11 208.5 | 500.5 | 331.9 | 297.1 | 281.4 | 219.9 | 2.4 | 2.4 |
| xz | 10 061.3 | 2 600.8 | 659.9 | 478.2 | 292.5 | 104.8 | 2.4 | 2.4 |

**Page fault rate (A).** Figure 2 plots the page fault rate against physical memory size. Once memory is large enough that nothing is evicted, the rate is constant for each benchmark: it is just the compulsory first-touch faults divided by the number of accesses (Table 5, last column). Below that point the fault rate increases as memory shrinks.

**When replacement starts (A).** In all 160 runs, a benchmark has zero evictions exactly when the number of frames is at least the number of pages it touches (Table 1). Once memory is smaller than that, the number of faults minus the number of evictions equals the number of frames, i.e. memory fills completely before the first eviction.

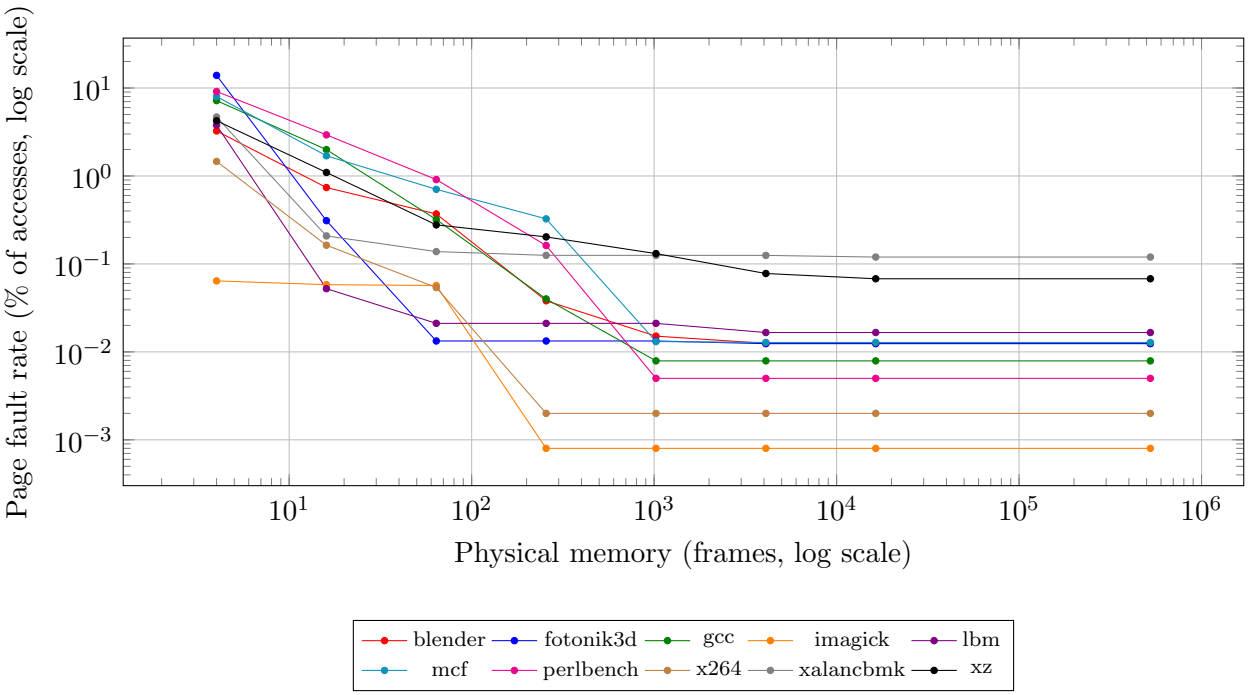



Figure 2: Page fault rate (faults per access, including first-touch faults, in percent) versus physical memory size (two-level table with TLB; identical for the single-level table). Each curve levels off at the right at the benchmark's compulsory fault rate, the number of pages it touches divided by its number of accesses.

**Cycles (B).** Wherever any eviction happens, the disk cost dominates: it accounts for at least 82.9% of the total cycles in every run with at least one eviction. As a result, total cycles rise by orders of magnitude as soon as memory drops below a benchmark's footprint (Table 6, Figure 4).

**Page table type.** In every one of the 80 (benchmark, size) pairs, the single-level and two-level simulators have identical fault counts, eviction counts and TLB hit rates; only the cycle counts differ, by the extra walk access. Replacement is decided by the frame manager and does not depend on the page table.

**Clean and dirty pages.** Every eviction costs the full 20 million cycles. The trace's read/write flags are ignored, so the simulator does not distinguish clean pages from dirty ones and does not model write-back: evicting a page that was only read costs the same as evicting one that was written. Under a model where clean pages can be dropped without a disk write, the cycle counts in Part IV would be lower, so they are an upper-end estimate under that alternative; we have not measured how much lower.

## 7 Part V: Simulator Speed

Table 7 gives the wall-clock time of the complete simulator (two-level page table with TLB, as in Part III) for each benchmark. The time is measured with a monotonic clock around constructing the simulator, reading the trace and simulating every access; it includes reading the trace from disk and excludes printing. Each figure is from one run, so it is subject to run-to-run noise. The ten benchmarks together take 4.0 s.

Design choices that affect speed: the page tables, the frame list, the per-page access counters and the TLB are all flat arrays (no hash maps or node-based containers); the LRU list and its updates are constant time; the TLB is a linear scan over only 128 entries; and the trace is read in

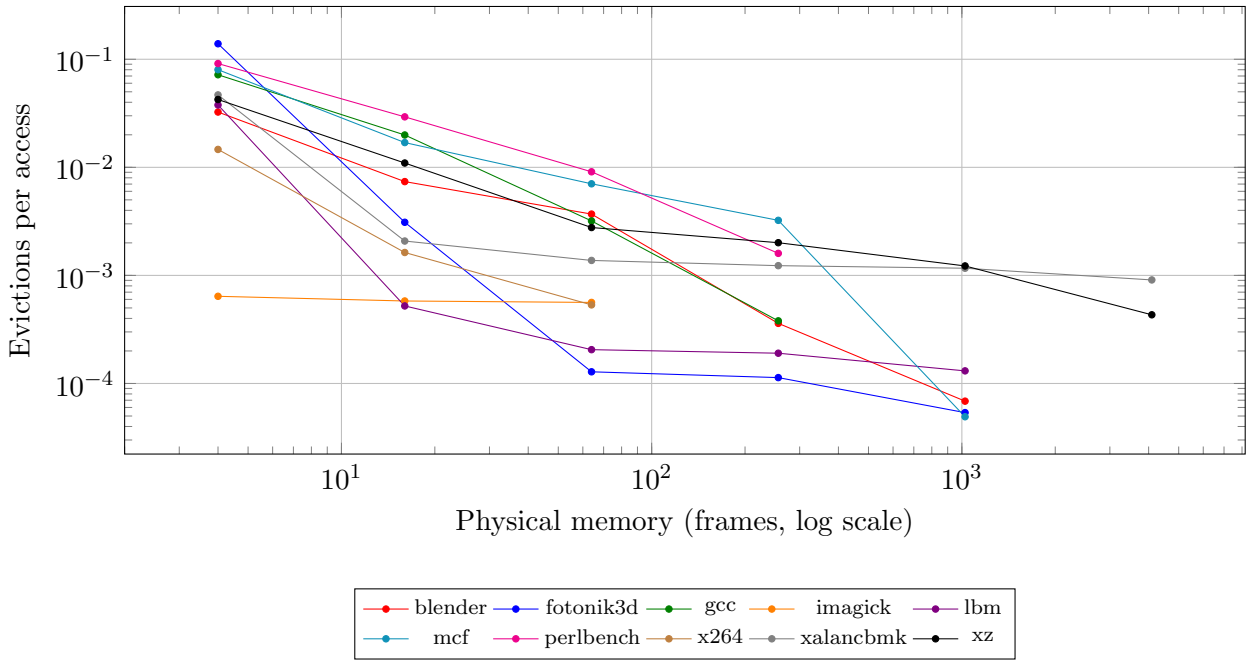



Figure 3: Evictions per access versus physical memory size (two-level table; identical for the single-level table). Runs with zero evictions cannot be drawn on a logarithmic axis and are omitted, so each curve starts at the first memory size where that benchmark evicts a page.

1 MB chunks and parsed by hand, with no per-line string handling. The programs are built with `-O2`.

Table 7: Simulator time per benchmark (two-level page table with TLB, `-O2`).

| Benchmark | Accesses (M) | Time (s) | M accesses/s |
|---|---|---|---|
| blender | 12.37 | 0.326 | 37.9 |
| fotonik3d | 12.89 | 0.285 | 45.2 |
| gcc | 12.47 | 0.572 | 21.8 |
| imagick | 11.55 | 0.213 | 54.2 |
| lbm | 12.84 | 0.266 | 48.3 |
| mcf | 12.47 | 0.506 | 24.7 |
| perlbench | 12.57 | 0.583 | 21.6 |
| x264 | 11.72 | 0.553 | 21.2 |
| xalancbmk | 11.97 | 0.277 | 43.2 |
| xz | 11.85 | 0.405 | 29.3 |

# 8 Notes and Limitations

- Page table utilization for the two-level table excludes the fixed 1024-entry directory. It is only reported for Parts I–III; in Part IV second-level tables can be freed by evictions, so the value at the end of a run is not comparable.
- "Page fault rate" in Part IV is faults per access including compulsory first-touch faults. The eviction rate (replacements per access) is also in `results/part_iv.csv`.
- Every eviction is charged the full 20 million cycles. The read/write flags of the trace are ignored, so there is no distinction between clean and dirty pages and no write-back modelling.

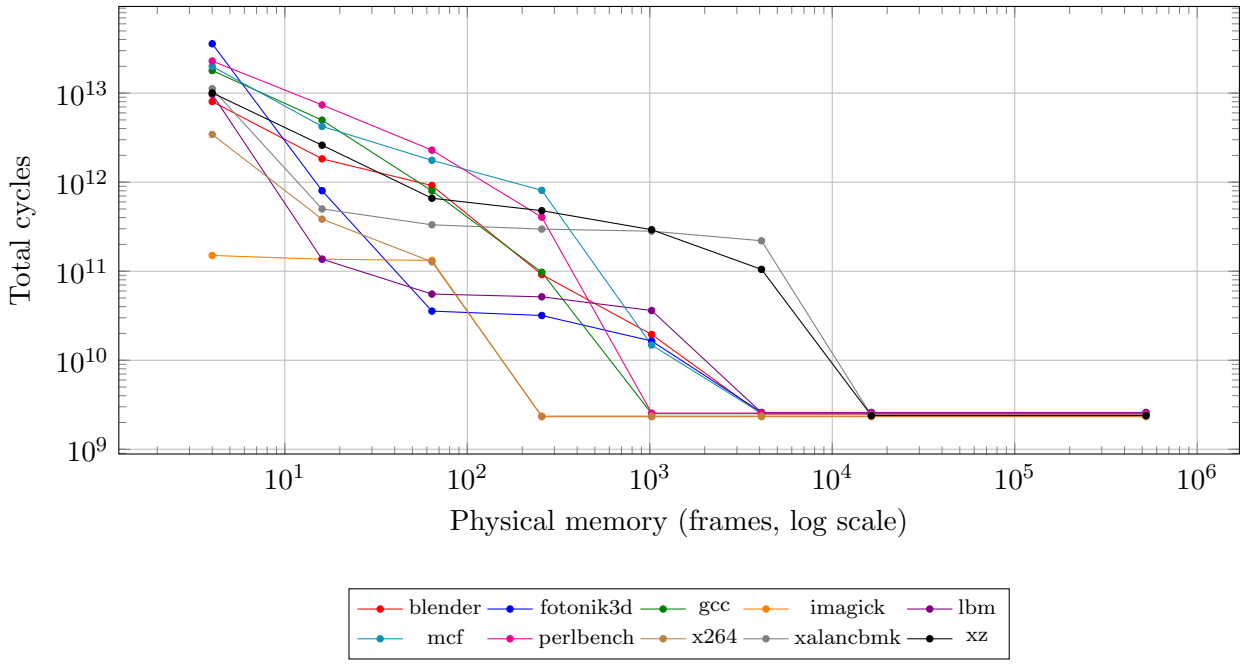



Figure 4: Total cycles versus physical memory size (two-level table with TLB).

- Page-table storage is not modelled as using physical frames, so it does not compete with the benchmark's pages for memory in Part IV.